



# Android 12~Android 13 SELinux 客制化指导手册

文档版本  
发布日期

V1.3  
2022-12-09

**版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。**

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

## 商标声明

**紫光展锐**、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

Bluetooth®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

# 紫光展锐（上海）科技有限公司



# 前言

## 概述

本文档详细地描述了 SELinux 模块基础知识、配置方法，提供了相关问题的解决方法。

## 读者对象

本文档主要适用于想了解 Android 12 和 Android 13 SELinux 相关功能和问题的研发与测试人员。

## 缩略语

| 缩略语     | 英文全名                         | 中文解释        |
|---------|------------------------------|-------------|
| AVC     | Access Vector Cache          | 访问向量缓存      |
| DAC     | Discretionary Access Control | 自主访问控制      |
| MAC     | Mandatory Access Control     | 强制访问控制      |
| SELinux | Security-Enhanced Linux      | 安全增强型 Linux |

## 符号约定

在本文中可能出现下列符号，每种符号的说明如下。

| 符号                                                                                            | 说明                                                    |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------|
|  <b>说明</b> | 用于突出重要或关键信息、补充信息和小窍门等。<br>“说明”不是安全警示信息，不涉及人身、设备及环境伤害。 |
|  <b>注意</b> | 用于突出容易出错的操作。<br>“注意”不是安全警示信息，不涉及人身、设备及环境伤害。           |
|  <b>警告</b> | 用于可能无法恢复的失误操作。<br>“警告”不是危险警示信息，不涉及人身及环境伤害。            |

## 变更信息

| 文档版本 | 发布日期       | 修改说明                                                                                                                                                                                                       |
|------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| V1.3 | 2022-12-09 | 刷新 <a href="#">3.1 优化目的</a> 描述。                                                                                                                                                                            |
| V1.2 | 2022-09-05 | 刷新 <a href="#">4.3 Neverallow 问题</a> 的步骤 2。                                                                                                                                                                |
| V1.1 | 2022-07-01 | <ul style="list-style-type: none"><li>新增支持 Android 13</li><li>新增 <a href="#">2.3 新增服务配置安全上下文</a></li><li>新增 <a href="#">3 Sepolicy 分区配置优化</a></li><li>新增 <a href="#">4.5 rc 中新增 service 常见报错</a></li></ul> |
| V1.0 | 2021-11-04 | 第一次正式发布。                                                                                                                                                                                                   |

## 关键字

SELinux、AVC、MAC

# 目 录

|                               |    |
|-------------------------------|----|
| 1 SELinux 基础知识 .....          | 1  |
| 1.1 访问控制 .....                | 1  |
| 1.2 SELinux 模式 .....          | 1  |
| 1.3 SELinux 模式查看和切换 .....     | 1  |
| 1.3.1 SELinux 模式查看 .....      | 1  |
| 1.3.2 SELinux 模式切换 .....      | 2  |
| 1.4 AVC 基础语法 .....            | 2  |
| 1.5 SELinux 策略 .....          | 3  |
| 2 SELinux 配置指导 .....          | 5  |
| 2.1 新增文件配置安全上下文 .....         | 5  |
| 2.1.1 新增设备节点文件 .....          | 5  |
| 2.1.2 新增 proc 文件 .....        | 5  |
| 2.2 新增属性配置安全上下文 .....         | 6  |
| 2.2.1 Android 属性介绍 .....      | 6  |
| 2.2.2 新增属性 .....              | 6  |
| 2.3 新增服务配置安全上下文 .....         | 7  |
| 2.3.1 配置介绍 .....              | 7  |
| 2.3.2 配置方法 .....              | 7  |
| 2.4 终端中查看安全上下文方法 .....        | 8  |
| 2.5 代码修改后单独编译验证 .....         | 8  |
| 2.6 SELinux 安全上下文恢复 .....     | 9  |
| 3 Sepolicy 分区配置优化 .....       | 10 |
| 3.1 优化目的 .....                | 10 |
| 3.2 优化原则 .....                | 11 |
| 4 SELinux 问题定位指导 .....        | 13 |
| 4.1 SELinux 权限问题判断 .....      | 13 |
| 4.2 添加 SELinux 权限 .....       | 13 |
| 4.3 Neverallow 问题 .....       | 14 |
| 4.4 mls 规则问题 .....            | 14 |
| 4.5 rc 中新增 service 常见报错 ..... | 15 |

## 图目录

|                                  |    |
|----------------------------------|----|
| 图 3-1 vendor 分区的策略编译产物 .....     | 10 |
| 图 3-2 system 分区的策略编译产物 .....     | 11 |
| 图 3-3 system_ext 分区的策略编译产物 ..... | 11 |

# 表目录

|                                  |    |
|----------------------------------|----|
| 表 3-1 SELinux 策略路径以及对应编译产物 ..... | 11 |
|----------------------------------|----|

# 1 SELinux 基础知识

## 1.1 访问控制

Linux 内核资源访问控制分为 DAC（Discretionary Access Control，自主访问控制）和 MAC（Mandatory Access Control，强制访问控制）两类。

### DAC

基于“用户-用户组-其他”的“读、写、执行”的权限检查，进程理论上所拥有的权限与执行它的用户的权限相同，该管理过于宽松，如果获得 root 权限，可以在 Linux 系统内做任何事情。

### MAC

SELinux（Security-Enhanced Linux，安全增强型 Linux）是 MAC 机制的一种实现，基于安全上下文和安全策略的安全机制，用于补充 DAC 检查。访问系统资源时，会先进行 DAC 检查，DAC 检查通过，才能进行 MAC 检查，如果 MAC 检查通过，才能获得资源访问权限。

## 1.2 SELinux 模式

SELinux 分为 Permissive 和 Enforcing 两种模式。

- Permissive  
宽容模式，代表 SELinux 运作中，违反 SELinux 规则，不会限制资源访问，只打印警告信息（avc: denied）到日志中。这种模式可用于 debug SELinux，查看无法访问的原因。
- Enforcing  
强制模式，代表 SELinux 运作中，违反 SELinux 规则的行为将被阻止，并打印警告信息（avc: denied）到日志中。

## 1.3 SELinux 模式查看和切换

### 1.3.1 SELinux 模式查看

- 通过命令行查看 SELinux 模式，命令如下：  

```
adb shell getenforce
```
- 通过 AVC（Access Vector Cache，访问向量缓存）log 来查看 SELinux 模式，AVC log 结尾会有 permissive=1/0 的标示：
  - permissive=1，说明是 Permissive 模式。

- permissive=0，说明是 Enforcing 模式。

### 1.3.2 SELinux 模式切换

默认为 Enforcing 模式，可通过命令行、修改 dts bootargs 参数或修改 init 代码切换成 Permissive 模式。

#### 命令行

终端开机后，开启终端 USB debug 选项，用 USB 线将终端与电脑连接，在电脑输入如下命令：

```
adb root
adb shell setenforce 0
```

#### 说明

- 可以使用 getenforce 查看 SELinux 模式。
- 该方式仅适用于 userdebug 版本，系统重启会导致修改失效。

#### 修改 dts bootargs 参数

board 对应的 dts 文件，在 bootargs 参数里增加 “androidboot.selinux=permissive” 字段，如在 kernel/common/arch/arm64/boot/dts/sprd/sl8541e\_1h10.dts 文件中，如下红色字体所示：

```
bootargs = "earlycon=sprd_serial,0x508d0000,115200n8 console=ttySE0,115200n8 loglevel=1 init=/init
root=/dev/ram0 rw androidboot.hardware=sl8541e_1h10 vmalloc=360M androidboot.dtbo_idx=0
printk.devkmsg=on androidboot.selinux=permissive androidboot.boot_devices=soc/soc:ap-ahb/20600000.sdio"
```

修改完成后重新编译代码，再将生成的 boot.img 烧写到终端。

#### 说明

该方式仅适用于 userdebug 版本，系统重启修改仍生效。

#### 修改 init 代码

修改 system/core/init/selinux.cpp 文件里的 IsEnforcing() 函数，将该函数直接返回 false 值，如下红色字体所示：

```
bool IsEnforcing() {
return false;
.....
}
```

修改完成后重新编译代码，再将生成的 super.img 烧写到终端。

#### 说明

该方式适用于 user 和 userdebug 版本，系统重启修改仍生效。

## 1.4 AVC 基础语法

AVC log 举例如下：

```
22:33:11.062 <36>[ 8.755284] c6 type=1400 audit(1325428390.909:11): avc: denied { read } for pid=1834  
comm="gnss_download" name="mmcblk0p17" dev="tmpfs" ino=10268 scontext=u:r:gnss_download:s0  
tcontext=u:object_r:block_device:s0 tclass=blk_file permissive=0
```

## AVC log 说明

- `avc: denied`: 表示当前操作被拒绝。
- `{ read }`: 表示被拒绝的操作, `{ }` 中含有实际尝试的操作。
- `for pid=1834`: 当前发生 `avc: denied` 进程的 ID。
- `comm= "gnss_download"`: 当前发生 `avc: denied` 进程的名称, 即主体进程名称。
- `name= "mmcblk0p17"`: 操作尝试的目标文件或目录的路径, 即客体资源名称。
- `dev= "tmpfs"`: 含有这个文件系统的设备节点, 客体资源在该文件系统中。
- `ino=10268`: 目标文件或目录的节点号。
- `scontext=u:r:gnss_download:s0`: 主体进程的安全上下文。
- `tcontext=u:object_r:block_device:s0`: 客体资源的安全上下文。
- `tclass=blk_file`: 访问资源所属类别。
- `permissive=0`: 当前是 Enforcing 模式, `permissive=1` 时为 Permissive 模式。

### 说明

此 AVC log 说明 `gnss_download` (进程) 缺少对标签为 `block_device`、类型为 `blk_file` 和名称为 `mmcblk0p17` 文件的 `read` 权限, 权限的添加可参考 [4.2 添加 SELinux 权限](#)。

## 1.5 SELinux 策略

### SELinux 策略类型

- **Public 公共策略**: 谷歌原生策略中的 `public` 部分以及开发人员自行添加策略中的 `public` 部分。
- **Private 私有策略**: 谷歌原生策略中的 `private` 部分以及开发人员自行添加策略中的 `private` 部分。
- **Vendor 供应商通用组件策略**: 谷歌原生策略中的 `vendor` 部分以及开发人员自行添加策略中的 `vendor` 部分。

### 谷歌原生策略

- `/system/sepolicy`: 谷歌原生策略, 不要在此路径下进行修改
- `/system/sepolicy/private`: 谷歌原生 `system` 分区 `private` `sepolicy`
- `/system/sepolicy/public`: 谷歌原生 `system` 分区 `public` `sepolicy`
- `/system/sepolicy/vendor`: 谷歌原生 `vendor` 分区 `sepolicy`
- `/system/sepolicy/prebuilts`: 版本兼容 `sepolicy`

### 展锐补充策略

- `device/sprd/mpool`: 展锐一般在此路径配置修改 SELinux 策略
- `device/sprd/mpool/sepolicy/system/private`: `system` 分区 `private` `sepolicy`
- `device/sprd/mpool/sepolicy/system/public`: `system` 分区 `public` `sepolicy`

- device/sprd/mpool/sepolicy/vendor: vendor 分区 sepolicy
- device/sprd/mpool/module/xxx/msepolicy/system/private: xxx 模块的 system 分区 private sepolicy
- device/sprd/mpool/module/xxx/msepolicy/system/public: xxx 模块的 system 分区 public sepolicy
- device/sprd/mpool/module/xxx/msepolicy/vendor: xxx 模块的 vendor 分区 sepolicy

# 2 SELinux 配置指导

## 2.1 新增文件配置安全上下文

新增文件配置主要包含虚拟文件配置写入 `genfs_contexts` 文件和磁盘文件配置写入 `file_contexts` 文件。

### 2.1.1 新增设备节点文件

以新增设备节点 `/dev/autotst` 为例，配置安全上下文步骤如下：

步骤 1 确认新增的设备节点类型是否在 `system/sepolicy/public/device.te` 文件中定义。

- a 已有定义，跳至[步骤 2](#)。
- b 没有定义，需要自定义类型，打开 `device/sprd/mpool/sepolicy/vendor/device.te` 文件，添加如下代码：

```
type autotest_device, dev_type;
```

步骤 2 `file_contexts` 中配置文件安全上下文。

打开 `device/sprd/mpool/sepolicy/vendor/file_contexts` 文件，添加如下代码：

```
/dev/autotst u:object_r:autotest_device:s0
```

步骤 3 `te` 文件中赋予对应进程访问该类型文件的权利。

打开 `device/sprd/mpool/sepolicy/vendor/autotest.te` 文件，添加如下代码：

```
allow autotest autotest_device:dir { search read };
```

----结束

#### 说明

本节主要是以 `vendor` 举例说明，`system` 的也可参考。

### 2.1.2 新增 `proc` 文件

`proc` 文件属于虚拟文件系统，例如蓝牙驱动进程会在开机过程中在 `proc` 下创建一个文件节点。为此文件节点配置安全上下文的步骤如下：

步骤 1 确认新增 `proc` 文件的类型是否为已有类型。

- a 是已有类型，跳至[步骤 2](#)。
- b 不是已有类型，则需重定义标签，打开 `device/sprd/mpool/sepolicy/vendor/file.te` 文件，添加如下代码：

```
type proc_bluetooth_writable, fs_type, proc_type;
```

步骤 2 `genfs_contexts` 中配置 `proc` 文件安全上下文。

打开 `device/sprd/mpool/sepolicy/vendor/genfs_contexts` 文件，添加如下代码（其中节点路径要写绝对路径，否则不生效）：

```
genfscon proc /bluetooth/sleep/btwrite u:object_r:proc_bluetooth_writable:s0
```

步骤 3 te 文件中赋予对应进程访问该类型文件的权利。

打开 `device/sprd/mpool/module/wcn/msepolicy/vendor/bluetooth.te` 文件，添加如下代码：

```
allow bluetooth proc_bluetooth_writable:file write;
```

----结束

## 说明

如需查看文件安全上下文配置情况，可参考 [2.4 终端中查看安全上下文方法](#)。

## 2.2 新增属性配置安全上下文

Android 属性 property 在系统中存放在一块共享内存中，每个进程都能读，但是只有 init 进程能够写，请求 init 进程帮助写属性值是需要 SELinux 权限的。

### 2.2.1 Android 属性介绍

#### Android 属性名称

必须用 `system/core/init/property_service.c` 中定义的前缀，例如 vendor 属性名前缀为如下 3 种：

- `persist.vendor.**`
- `vendor.**`
- `ro.vendor.**`

以上 vendor 域属性，所打标签必须以 `vendor_` 开头，否则会导致 XTS 测试失败。

### 2.2.2 新增属性

根据属性的命名规范，选择定义在 system 分区还是 vendor 分区，以 vendor 为例，新增属性步骤如下：

步骤 1 在 `property.te` 中添加属性的 type 定义。

打开 `device/sprd/mpool/sepolicy/vendor/property.te` 文件，添加如下代码：

```
type vendor_yyy_prop, property_type, vendor_property_type;
```

#### 注意

定义 label 时，需要使用“`type xxx property_type, system_property_type` 或 `vendor_property_type` 或 `product_property_type`”作为必须项目（`product_property_type` 几乎不用）。如果这样的定义触发 Neverallow，则追加关联 `xxx_public_property_type`。如“`type vendor_yyy_prop, property_type, vendor_property_type, vendor_public_property_type`”。

步骤 2 `property_contexts` 中对属性设定安全上下文。

打开 `device/sprd/mpool/sepolicy/vendor/property_contexts` 文件，添加如下代码：

```
xxx.xxx. u:object_r:vendor_yyy_prop:s0
```

### 说明

xxx.xxx 后的“.”表示匹配以 xxx.xxx 开头的属性。

步骤 3 te 文件中赋予对应进程访问该类型属性的权利。

打开 device/sprd/mpool/module/app/msepolicy/vendor/xxx.te 文件，添加如下代码：

```
set_prop(xxx, vendor_yyy_prop)
```

### 说明

添加属性权限时，推荐使用 getprop 和 setprop 宏。

----结束

### 说明

如需查看文件安全上下文配置情况，可参考 [2.4 终端中查看安全上下文方法](#)。

## 2.3 新增服务配置安全上下文

### 2.3.1 配置介绍

- 在 rc 文件中增加 service 进程（服务进程），绝大多数 service 进程由 init 进程启动。
- 只有 init 进程的安全上下文能配置为 u:r:init:s0，同时 XTS 测试中会校验配置安全上下文为 u:r:init:s0 的进程个数，如果大于 1 个，则测试失败。
- 从 init 进程启动的 service 进程必须配置自己的安全上下文（若未配置会继承 init 进程的安全上下文 u:r:init:s0）。
  - 在 Android 5.1 版本，若不定义安全上下文，XTS 测试失败。
  - 在 Android 6.0 及之后版本，若不定义安全上下文，user 版本的 service 进程无法启动。
- rc 文件中的 service 有时又称 native service，常常添加于如下文件中：/system/core/init.rc 和 /device/sprd/xxx/各种.rc 文件。

### 2.3.2 配置方法

新增 native service 示例步骤如下。

步骤 1 按照 push 流程，把想要启动的文件 push 到终端中。

步骤 2 rc 文件中添加 bin 文件。

```
service watchlssd /system/bin/watchlssd
```

步骤 3 添加 te 文件（例：watchlssd.te）。

- private/watchlssd.te

```
+typeattribute watchlssd coredomain;
+init_daemon_domain(watchlssd) //init_daemon_domain宏的功能是：init进程执行一个type
为watchlssd_exec的文件，创建一个子进程。
```
- public/watchlssd.te

```
+type watchlssd, domain;    //本进程domain是watchlssd。

+type watchlssd_exec, exec_type, file_type;    //watchlssd_exec将会设置为
/system/bin/watchlssd的type，即/system/bin/watchlssd文件的安全上下文（这两种type可能不全，如果编译出错，在这两种type后加上编译报错中提示缺少的type再编译，逗号后有空格且最后分号结尾）。进程启动时，将该子进程的domain设置为watchlssd，而不是继承父进程的init。

+allow watchlssd xxx;    //为watchlssd申请权限，使用权限最小原则，只申请需要的权限。
```

步骤 4 在 file\_contexts 文件中赋予/system/bin/watchlssd 可执行文件安全上下文。

```
/system/bin/watchlssd    u:object_r:watchlssd_exec:s0
```

步骤 5 重新整编版本烧写到终端后生效。

----结束

## 2.4 终端中查看安全上下文方法

- 可通过如下指令查看文件安全上下文：

```
ls -lZ
```

- 可通过如下指令查看属性安全上下文：

```
getprop -Z
```

- 可通过如下指令查看进程安全上下文：

```
ps -Z
```

## 2.5 代码修改后单独编译验证

如果只是修改 SELinux 相关文件，可以参考如下步骤进行快速验证：

步骤 1 lunch 工程

步骤 2 输入如下命令单独编译 sepolicy 模块

```
cd system/sepolicy
mma
```

步骤 3 判断编译结果

- 编译成功表示修改正确。
- 编译失败表示修改错误。

----结束

## 2.6 SELinux 安全上下文恢复

SELinux 安全上下文的恢复可以使用 `restorecon` 命令来实现，格式如下：

```
[root@localhost ~] # restorecon [选项] 文件或目录
```

选项：

- **-R**：递归，当前目录和目录下所有的子文件同时恢复。
- **-V**：恢复过程详细信息显示到屏幕上。

例如，想要恢复 `file_contexts` 中的安全上下文配置，可使用如下指令恢复标签：

```
restorecon /mnt/vendor
```

- 如果在终端上使用 `restorecon` 命令可以成功恢复正确标签，则可在所需函数或 `rc` 文件中添加 `restorecon` 命令恢复标签，以此完善函数或 `rc` 的功能。

```
restorecon -R 文件路径
```

- 如果在终端上使用 `restorecon` 命令不能恢复成功，则需要抓取 `log` 查看是否缺少相关权限，继而添加相应权限。

# 3 Sepolicy 分区配置优化

## 3.1 优化目的

sepolicy 分区优化后，代码编译产物与代码运行所需 sepolicy 属于同一分区，顺应了谷歌的 SELinux 趋势。

SELinux 策略分为 vendor 域和 system 域。

- vendor 域包含了 vendor 分区和 odm 分区的 SELinux 策略。
- system 域包含了 system 分区、product 分区和 system\_ext 分区的 SELinux 策略。

### Vendor 域 SELinux 策略

在 out/target/product/xxx/vendor/etc/selinux 目录的 cil 文件中可以看到 vendor 域的 SELinux 策略，如图 3-1 所示。

- 在 system 域定义的类型，在 vendor 域被编译时都会带上 sepolicy\_version。
  - cmd\_services 的名称为 cmd\_services\_30\_0，这是因为 cmd\_services 的定义在 system 域的 public 目录中。
  - init 的名称为 init\_30\_0，因为其定义也是在 system 域。
- 在 vendor 域定义的类型，在 vendor 域被编译时不会带上 sepolicy\_version。
  - mmcblk\_device 的名称没有携带 sepolicy\_version，因为其定义在 vendor 域。

带上 sepolicy\_version 后，cmd\_services 的所有 allow 将无法被识别。将 cmd\_services 的策略（allow xxx）与其定义一起放在 system 域的 public 目录，则 allow 可以识别。

图3-1 vendor 分区的策略编译产物

```
(allow cmd_services_30_0 ctb_device (chr_file (ioctl read open)))  
(allow cmd_services_30_0 init_30_0 (unix_stream socket (connectto)))  
(allow cmd_services_30_0 ctt_default_prop_30_0 (property_service (set)))  
(allow cmd_services_30_0 system_prop_30_0 (property_service (set)))  
(allow cmd_services_30_0 block_device_30_0 (dir (search)))  
(allow cmd_services_30_0 mmcblk_device (blk_file (read open)))
```

### System 域 SELinux 策略

在 out/target/product/xxx/system\_ext/etc/selinux/ 目录的 cil 文件中可以看到 system 域的 SELinux 策略，如图 3-2 和图 3-3 所示。

图3-2 system 分区的策略编译产物

```
1 (allow traced_probes system_file (dir (read open)))
2 (allow traced_probes self (capability (dac_read_search)))
3 (allow traced_probes self (cap_userns (dac_read_search)))
4 (allow traced_probes apk_data_file (dir (read getattr open search)))
5 (allow traced_probes dalvikcache_data_file (dir (read getattr open search)))
6 (allow traced_probes system_data_file (dir (read open)))
7 (allow traced_probes system_app_data_file (dir (read getattr open search)))
8 (allow traced_probes backup_data_file (dir (read getattr open search)))
9 (allow traced_probes bootstat_data_file (dir (read getattr open search)))
```

图3-3 system\_ext 分区的策略编译产物

```
(allow init modemlog_connmgr_service_exec (file (read getattr map execute open)))
(allow init modemlog_connmgr_service (process (transition)))
(allow modemlog_connmgr_service modemlog_connmgr_service_exec (file (read getattr map execute open)))
(dontaudit init modemlog_connmgr_service (process (noatsecure)))
(allow init modemlog_connmgr_service (process (siginh rlimitinh)))
(typetransition init modemlog_connmgr_service_exec process modemlog_connmgr_service)
(allow init slogmodem_exec (file (read getattr map execute open)))
(allow init slogmodem (process (transition)))
(allow slogmodem slogmodem_exec (file (read getattr map execute open)))
(dontaudit init slogmodem (process (noatsecure)))
(allow init slogmodem (process (siginh rlimitinh)))
(typetransition init slogmodem_exec process slogmodem)
```

## 3.2 优化原则

策略路径以及对应编译产物的名称如表 3-1。

表3-1 SELinux 策略路径以及对应编译产物

| 序号 | 路径                                        | 对应编译产物                  |
|----|-------------------------------------------|-------------------------|
| 1  | device/sprd/mpool/sepolicy/system/private | system_ext_speolicy.cil |
| 2  | device/sprd/mpool/sepolicy/system/public  | system_ext_speolicy.cil |
| 3  | device/sprd/mpool/sepolicy/vendor         | vendor_sepolicy.cil     |
| 4  | system/sepolicy/public                    | plat_sepolicy.cil       |
| 5  | system/sepolicy/private                   | plat_sepolicy.cil       |
| 6  | system/sepolicy/vendor                    | plat_pub_versioned.cil  |

优化原则参考如下场景。

### 场景 1：system 与 vendor 跨域访问

- 策略（allow）位置

Zygote（system）为 logmanager（vendor）创建一个服务。zygote 访问 logmanager.bin 的策略，需放在 system/public 目录，使其编译至 system\_ext 分区中，与 zygote 同属于 system\_ext 分区。

logmanager 运行过程中如读取系统文件、系统属性的策略（allow），需放在 vendor 目录，使其编译至 vendor 分区，与 logmanager.bin 同属于 vendor 分区。

- 标签定义（type）位置

由于是 system 与 vendor 跨域访问，标签定义应存放于 system/public 目录。

- 标签配置（contexts）位置

file\_contexts、property\_contexts、service\_contexts 跟随文件、属性、服务对应可执行文件的编译去向。

- 系统文件、属性、服务标签配置放在 system/public 目录
- vendor 文件、属性、服务标签配置放在 vendor 目录
- genfs\_context 中存放虚拟文件系统标签，推荐全部放在 vendor 目录

## 场景 2：system 域内部访问

- 策略（allow）位置

放在 system/public 或 system/private 目录，尽可能放在 system/public 目录。

- 标签定义（type）位置

放在 system/public 或 system/private 目录，尽可能放在 system/public 目录。

- 标签配置（contexts）位置

放在 system/public 或 system/private 目录，尽可能放在 system/public 目录。

## 场景 3：vendor 域内部访问

- 策略（allow）位置

放在 vendor 目录。

- 标签定义（type）位置

放在 vendor 目录。

- 标签配置（contexts）位置

放在 vendor 目录。

# 4 SELinux 问题定位指导

## 4.1 SELinux 权限问题判断

当在 Enforcing 模式下出现代码执行异常时，可同步在 Permissive 模式下验证该异常是否依旧存在：

- 如果异常消失，则说明该异常与 SELinux 权限问题有关，可参考 [4.2 添加 SELinux 权限](#) 进行相应权限的添加。
- 如果异常仍然存在，则说明该异常与 SELinux 权限问题无关，需排查其它原因。

## 4.2 添加 SELinux 权限

由于缺少 SELinux 权限导致如下 “avc: denied”，需要根据 AVC log 信息添加相应权限。

```
avc: denied { read write } for pid=3483 comm="Binder_6" name="rtc0" dev="tmpfs" ino=4451  
scontext=u:r:system_server:s0 tcontext=u:object_r:refnotify_device:s0 tclass=chr_file permissive=0
```

### AVC log 含义

主体标签为 system\_server 的 Binder\_6 进程缺少对客体标签为 refnotify\_device、类型为 chr\_file 和名称为 rtc0 文件的读写权限。

### 添加权限参考格式

打开 device/sprd/mpool/sepolicy/xxx/scontext.te 文件，添加如下代码：

```
allow scontext tcontext:tclass permission;
```

#### 说明

xxx 为策略的放置路径，从 system/private、system/public 或 vendor 中进行选择。

### 添加权限

打开 device/sprd/mpool/sepolicy/system/private/system\_server.te 文件，添加如下代码：

```
allow system_server refnotify_device:chr_file { read write };
```

#### 说明

- 可以设置 SELinux 为 permissive 模式，使所有相关 AVC 暴露，添加权限。
- 在文件内做修改时，无论 te 文件还是 contexts 文件，在后面都要保留一个空行。

## 4.3 Neverallow 问题

### 【问题现象】

在添加如下代码进行编译时，编译失败并报 neverallow 错。

```
allow system_app sysfs:file {write};
```

### 【问题分析】

原因是谷歌不允许应用进程写 sysfs 类型的文件，其代码如下：

```
neverallow { appdomain -bluetooth -nfc }sysfs:dir_file_class_set write;
```

### 【解决方法】

这种情况下需要自定义 type，并更改客体的安全上下文。根据操作文件类型选择放入 vendor 或 system 文件夹下，下面以 vendor 举例，步骤如下：

步骤 1 参考原来的 sysfs 类型定义一种新 type，type 名字可以自定义。

打开 device/sprd/mpool/sepolicy/vendor/file.te 文件，添加如下代码：

```
type sysfs_xxx, fs_type,sysfs_type;
```

步骤 2 使用新定义的 type 为目标文件配置安全上下文。

打开 device/sprd/mpool/sepolicy/vendor/genfs\_contexts 文件，添加如下代码：

```
/xxx/xxx（目标文件路径） u:object_r:sysfs_xxx:s0
```

步骤 3 重新赋予主体进程访问新类型文件的权限。

打开 device/sprd/mpool/sepolicy/vendor/system\_app.te 文件，添加如下代码：

```
allow system_app sysfs_xxx:file { write };
```

----结束

### 📖 说明

Android 12 和 Android 13 中的 neverallow 规则比之前版本的要更加严格。除了访问部分属性，所有跨域的访问（vendor 进程访问 system 资源或者 system 进程访问 vendor 资源）全被拒绝。若属性跨域，必须关联 xxx\_public\_property\_type，详细请参考 [2.2.2 新增属性](#)。

## 4.4 mls 规则问题

如果 te 文件已经添加 SELinux 权限，但没有生效，查看 AVC log 信息出现“s0:c512,c768”字样，则可判断是由于 mls 规则导致。说明主体和客体安全级别不同，可通过使主体进程关联 mlstrustedsubject 或客体关联 mlstrustedobject 来解决。但是如果主客体之间的操作是 connectto，这种情况与上种情况略有不同，主体或者客体都要关联 mlstrustedsubject。

## 【问题现象】

已经在 platform\_app.te 中添加了 SELinux 权限，但 log 中依然有如下报错：

```
avc: denied { connectto } for pid=2002 comm="slogHandlerThre" path=00736C6F676D6F64656D  
scontext=u:r:platform_app:s0:c512,c768 tcontext=u:r:slogmodem:s0 tclass=unix_stream_socket permissive=0
```

## 【问题分析】

这是因为谷歌在文件 system/sepolicy/private/mls 中进行了安全级的限制，限制代码如下：

```
#Stream connect: Client must be equivalent to server unless one of them is trusted.  
  
mlsconstrain unix_stream_socket { connectto }  
  
(l1 eq l2 or t1 == mltrustedsubject or t2 == mltrustedsubject);
```

## 【解决方法】

这种情况需客体标签关联 mltrustedsubject，这里 platform\_app 最好不要修改，所以要修改客体 slogmodem。具体修改方法如下：

打开 device/sprd/mpool/module/log/msepolity/system/public/slogmodem.te 文件，添加如下代码：

```
type slogmodem, domain, mltrustedsubject;
```

# 4.5 rc 中新增 service 常见报错

## 【问题现象】

添加 service，push 到终端验证，service 无法正常启动。开机失败，log 如下：

```
00EF4 <11> [ 4.994631][03-08 22:21:38.994] init: Could not start service 'vendor.sprd.hardware.vdsp@1.0-  
service' as part of class 'hal': File /vendor/bin/hw/vendor.sprd.hardware.vdsp@1.0-service(labeled  
"u:object_r:vendor_file:s0") has incorrect label or no domain transition from u:r:init:s0 to another SELinux domain  
defined. Have you configured your service correctly? https://source.android.com/security/selinux/device-  
policy#label\_new\_services\_and\_address\_denials
```

## 【问题分析】

正常编译的时候会设置标签，标签正确不会出现以上报错。采用 push 的场景需要手动执行恢复标签的命令。

## 【解决方法】

步骤 1 执行下面命令恢复 SELinux 标签。

```
adb shell restorecon -R /vendor/bin/hw/vendor.sprd.hardware.vdsp@1.0-service
```

步骤 2 重启终端。

----结束